

Magnitude, Wanda & 2:4 Sparsity, Worked Out by Hand

THREE SCORES, ONE BLOCK — AND WHY $|W| \times \|X\|$ IS SECRETLY A HESSIAN

What this document does. It scores one row of eight weights three ways — by magnitude $|W|$, by the Wanda score $|W| \cdot \|X\|$, and by a diagonal-Hessian impact — and then applies a **2:4 structured mask** (exactly two of every four weights zeroed) under each score. Every number comes from a short NumPy script (Appendix A, `m06_t02_magnitude_wanda_24_engine.py`). By the end you will see, on paper, a concrete weight that magnitude pruning throws away but Wanda keeps, and the one-line algebra showing Wanda² *equals* the diagonal approximation of SparseGPT’s impact score from Topic 1.

The one idea. A weight’s effect on the output is the weight *times the input it multiplies*. A tiny weight sitting on a loud, high-variance input moves the output more than a large weight on a silent input. Magnitude is blind to the input; Wanda multiplies it back in — which is exactly what the Hessian does.

CONCEPTS COVERED, IN ORDER

the magnitude score and its blind spot; the input activation norm $\|X_{:,j}\|_2$; the Wanda score; why Wanda $\approx$ Hessian importance (diagonal H^{-1}); the 2:4 structured-sparsity rule; applying a 2:4 mask to a block; and the $2\times$ tensor-core speedup it unlocks.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The block under study	2
2	Step 1 — The magnitude score (and its blind spot)	2
3	Step 2 — The input activation norm	2
4	Step 3 — The Wanda score	3
5	Step 4 — Why Wanda <i>is</i> a Hessian score	3
6	Step 5 — The 2:4 structured-sparsity rule	4
7	Step 6 — Apply the 2:4 mask to the block	4
8	Step 7 — What the hardware gets	5
9	The argument, at a glance	5
9.1	Equation cheat-sheet	5

Appendix A — Reference implementation

6

1 The block under study

We take a single output row of 8 weights (the input dimension), which is two 2:4 groups of four, plus the calibration activations that feed it.

Quantity	Symbol	Value here
Weight row	w	(0.42, 0.90, −0.31, 0.05, 0.60, 0.12, −0.80, 0.20)
Calibration set	X	6 inputs × 8 features
Sparsity pattern	—	2:4 (keep 2, zero 2, per group of 4)

The eight features split into **group 1** (features 0–3) and **group 2** (features 4–7). Within each group the 2:4 rule forces us to keep exactly two weights and zero the other two.

Intuition

Why $|W|$ alone is naive: imagine feature 2 carries a strong, always-present signal (its column of X has a big norm) but its weight is only −0.31. That weight, multiplied by a large input every time, contributes a sizeable, consistent term to the output. Magnitude pruning sees “0.31, small, drop it” and silently deletes a load-bearing connection. Wanda asks “0.31 times *what?*” first.

2 Step 1 — The magnitude score (and its blind spot)

The oldest pruning score, Optimal Brain Damage (1989) onward:

Magnitude importance

$$\text{score}_{\text{mag}}(j) = |w_j|.$$

For our row,

$$|w| = (0.42, 0.90, 0.31, 0.05, 0.60, 0.12, 0.80, 0.20).$$

In group 1 (features 0–3) the two largest are $|w_1| = 0.90$ and $|w_0| = 0.42$, so magnitude keeps $\{f_0, f_1\}$ and prunes $\{f_2, f_3\}$. Hold that thought — we are about to see it discard the wrong weight.

3 Step 2 — The input activation norm

Wanda needs one extra ingredient: how “loud” each input feature is across the calibration set. That is the L_2 norm of the corresponding *column* of X :

$$\|X_{:,j}\|_2 = \sqrt{\sum_n X_{nj}^2}.$$

From the six calibration rows:

$$\|X_{:,j}\|_2 = (2.470, 0.480, 7.166, 1.166, 1.127, 5.954, 0.714, 1.368).$$

Feature 2 is by far the loudest (7.17); feature 5 next (5.95). Those are precisely the columns where a small weight can still do a lot of work.

4 Step 3 — The Wanda score

Wanda (Sun et al., 2023) multiplies magnitude by activation norm:

Wanda importance — weight magnitude $\times$ input norm

$$\text{score}_{\text{Wanda}}(j) = |w_j| \cdot \|X_{:,j}\|_2.$$

Element by element:

$$\text{Wanda} = (1.037, 0.432, 2.221, 0.058, 0.676, 0.715, 0.571, 0.274).$$

Mini-example — group 1, weight by weight

feature	w_j	$ w_j $	$\ X_{:,j}\ $	Wanda
f_0	+0.42	0.42	2.47	1.037
f_1	+0.90	0.90	0.48	0.432
f_2	−0.31	0.31	7.17	2.221
f_3	+0.05	0.05	1.17	0.058

By magnitude, f_1 (0.90) looks like the most important weight in the group. By Wanda, f_1 drops to *third* — its input is quiet ($\|X\| = 0.48$), so $0.90 \times 0.48 = 0.432$. Meanwhile f_2 , dismissed by magnitude as small (0.31), is the *most* important by Wanda because it rides the loudest input.

Magnitude and Wanda keep different weights

Group 1, top-2 to keep: magnitude keeps $\{f_0, f_1\}$; Wanda keeps $\{f_0, f_2\}$. They disagree on the second slot. Magnitude prunes f_2 — the single most output-relevant weight in the group. Wanda prunes f_1 instead, whose large value never gets amplified by its (quiet) input. On a real layer, thousands of such disagreements are why Wanda loses only ~ 1 –3 MMLU points at 50% sparsity while magnitude loses 5–9.

5 Step 4 — Why Wanda *is* a Hessian score

This is the part worth carrying home. Recall Topic 1: SparseGPT ranks weights by

$$\text{impact}(i, j) = \frac{w_{ij}^2}{[H^{-1}]_{jj}}, \quad H = X^\top X.$$

The expensive part is inverting H . Wanda makes one approximation: treat H as *diagonal* (ignore cross-feature correlations). Then

$$H_{jj} = \|X_{:,j}\|_2^2, \quad [H^{-1}]_{jj} \approx \frac{1}{H_{jj}} = \frac{1}{\|X_{:,j}\|_2^2}.$$

Wanda² = diagonal-Hessian impact

Substitute the diagonal inverse into the impact score:

$$\text{impact}(j) \approx \frac{w_j^2}{1/\|X_{:,j}\|_2^2} = w_j^2 \|X_{:,j}\|_2^2 = (|w_j| \cdot \|X_{:,j}\|_2)^2 = \text{score}_{\text{Wanda}}(j)^2.$$

Since squaring is monotone increasing on non-negative numbers, *ranking by Wanda is identical to ranking by the diagonal-Hessian impact*.

The script confirms it numerically — the vector $w_j^2 \|X_{:,j}\|^2$ equals the vector Wanda^2 exactly:

$$w^2 \odot \text{diag}(H) = (1.076, 0.186, 4.935, 0.003, 0.457, 0.511, 0.326, 0.075) = \text{Wanda}^2. \quad \checkmark$$

The one-line summary of Wanda

Wanda is SparseGPT with the Hessian forced diagonal: no matrix inverse, no Cholesky, no error compensation — just $|W| \cdot \|X\|$ and a top- k keep. It captures the *first-order* reason a weight matters (its input is loud) and drops only the *second-order* cross-feature corrections. That is why it lands within a point of SparseGPT at a third of the code and runtime.

6 Step 5 — The 2:4 structured-sparsity rule

Everything above chose *how many* to prune by a quality budget. 2:4 fixes the count by hardware decree:

2:4 structured sparsity

In every group of 4 consecutive weights along the input dimension, *exactly 2 are zero and 2 are non-zero*. Sparsity is locked at 50%.

The reason is silicon: NVIDIA Ampere/Hopper sparse tensor cores store, per 4-group, the 2 surviving values plus a 2-bit index of where they sat, and run a “sparse GEMM” that reads half the bytes and issues half the multiply-accumulates — a hardware $2\times$. Unstructured sparsity at any ratio gets *zero* speedup on these kernels, because a dense kernel cannot skip arbitrarily placed zeros.

Watch out

2:4 is not “50% sparse on average.” It is 50% sparse in *every single group of four*. You cannot bank surplus zeros from a sparse region to keep an extra weight in a dense one. That rigidity is the price of the hardware speedup, and it is why 2:4 sometimes costs a touch more accuracy than free 50% unstructured.

7 Step 6 — Apply the 2:4 mask to the block

For each group of four, keep the top-2 by score, zero the rest.

	Group 1 (f_0 – f_3)				Group 2 (f_4 – f_7)			
weight w	0.42	0.90	−0.31	0.05	0.60	0.12	−0.80	0.20
Wanda score	1.04	0.43	2.22	0.06	0.68	0.71	0.57	0.27
mag 2:4 mask	✓	✓	.	.	✓	.	✓	.
Wanda 2:4 mask	✓	.	✓	.	✓	✓	.	.

The resulting pruned rows:

magnitude 2:4: (0.42, 0.90, 0, 0, 0.60, 0, −0.80, 0),
Wanda 2:4: (0.42, 0, −0.31, 0, 0.60, 0.12, 0, 0).

They differ in *both* groups. In group 1 Wanda rescues f_2 (loud input); in group 2 magnitude keeps $f_6 = -0.80$ (large weight, but quiet input $\|X\| = 0.71$), while Wanda keeps $f_5 = 0.12$ instead, whose input is loud ($\|X\| = 5.95$, Wanda = $0.715 > 0.571$).

8 Step 7 — What the hardware gets

Account for one 4-group stored in the 2:4 compressed format, FP16:

Format	Bytes/group	Note
Dense	$4 \times 2 = 8$	four FP16 weights
2:4 sparse	$2 \times 2 + 1 = 5$	two FP16 values + 2×2-bit index byte

The headline win is not the byte count of the group ($8 \rightarrow 5$); it is the **kernel**: the sparse tensor core performs half the multiply-accumulates and streams half the weight values, for a $2\times$ theoretical and $1.8\text{--}1.95\times$ measured throughput on H100 LLM decode. Because decode is memory-bandwidth-bound (Module 01), reading $\sim$ half the weight bytes per token translates almost directly into $\sim$ half the time-per-output-token. ✓

Intuition

2:4 is the rare “free” speedup: you do not change the math the model does (it still multiplies inputs by weights), you just guarantee the zeros land in a pattern the silicon was *built* to skip. Wanda or SparseGPT decides *which* two per group to drop; the hardware does the rest.

9 The argument, at a glance

#	Step	Result
1	Magnitude	$ w $; group 1 keeps $\{f_0, f_1\}$
2	Input norm	$\ X_{:,2}\ = 7.17$ is loudest
3	Wanda	$ w \ X\ $; group 1 keeps $\{f_0, f_2\}$
4	Hessian link	$\text{Wanda}^2 = w^2 \text{diag}(H)$ ✓
5	2:4 rule	exactly 2 of 4 zero; HW $2\times$
6	Apply mask	mag and Wanda differ in both groups
7	Speedup	$8 \rightarrow 5$ B/group, $1.8\text{--}1.95\times$ decode

9.1 Equation cheat-sheet

Magnitude score	$\text{score}_{\text{mag}}(j) = w_j $
Activation norm	$\ X_{:,j}\ _2 = \sqrt{\sum_n X_{nj}^2}$
Wanda score	$\text{score}_{\text{Wanda}}(j) = w_j \ X_{:,j}\ _2$
Diagonal-Hessian impact	$w_j^2 \ X_{:,j}\ _2^2 = \text{Wanda}^2$
2:4 rule	keep top-2 of every 4 by score
2:4 storage	$2 \times 2\text{B} + 1\text{B}$ index per 4-group

Appendix A — Reference implementation

Every number above is produced by `m06_t02_magnitude_wanda_24_engine.py`. Run it to reproduce the scores, masks, and the Wanda-equals-diagonal-Hessian check.

```

1  """
2  Reference implementation for: Module 06 - Topic 2
3  "Magnitude vs Wanda scoring, and the 2:4 structured-sparsity mask."
4
5  Three things, all on the SAME tiny weight block so the rankings can be compared
6  directly:
7      1. magnitude score |W[i,j]|
8      2. Wanda score      |W[i,j]| * ||X[:,j]||_2 (weight x input activation norm)
9      3. a 2:4 mask: in each group of 4 consecutive weights along the input dim,
10         keep exactly the top-2 by score, zero the other 2.
11
12  We use one output row of 8 weights (two 2:4 groups) plus the per-column input
13  activation norms, then show how magnitude and Wanda disagree about which two to
14  keep in a group, and how 2:4 differs from a free (unstructured) top-4 keep.
15  """
16
17  import numpy as np
18  np.set_printoptions(precision=4, suppress=True)
19
20  # -----
21  # 0. One output row of 8 weights = two 2:4 groups. Plus calibration X.
22  # -----
23  # w[j] = weight on input feature j.
24  w = np.array([0.42, 0.90, -0.31, 0.05, 0.60, 0.12, -0.80, 0.20])
25  # Calibration activations: N=6 rows, 8 columns (one per input feature).
26  # Some features are "loud" (large norm) even where the weight is small.
27  X = np.array([
28      [1.0, 0.2, 3.0, 0.5, 0.4, 2.5, 0.3, 0.6],
29      [0.8, 0.1, 2.5, 0.6, 0.5, 2.0, 0.2, 0.5],
30      [1.2, 0.3, 3.2, 0.4, 0.3, 2.8, 0.4, 0.7],
31      [0.9, 0.2, 2.8, 0.5, 0.6, 2.2, 0.3, 0.4],
32      [1.1, 0.1, 3.1, 0.3, 0.4, 2.6, 0.2, 0.6],
33      [1.0, 0.2, 2.9, 0.5, 0.5, 2.4, 0.3, 0.5],
34  ])
35  n_feat = w.shape[0]
36  print("== 0. Setup ==")
37  print("weights w =", w)
38
39  # -----
40  # 1. Per-column input activation L2 norm ||X[:,j]||_2.
41  # -----
42  col_norm = np.linalg.norm(X, axis=0)
43  print("\n== 1. Input activation norms ||X[:,j]||_2 ==")
44  print("||X[:,j]|| =", col_norm)
45
46  # -----
47  # 2. Magnitude score and Wanda score.
48  # -----
49  mag_score = np.abs(w)
50  wanda_score = np.abs(w) * col_norm
51  print("\n== 2. Scores ==")
52  print("magnitude |w| =", mag_score)
53  print("Wanda |w|*||X[:,j]|| =", wanda_score)
54
55  # -----
56  # 3. 2:4 mask under each score: per group of 4, keep top-2.
57  # -----

```

```

58 def mask_24(score):
59     m = np.zeros_like(score, dtype=int)
60     for g in range(0, len(score), 4):
61         grp = score[g:g+4]
62         keep = np.argsort(grp)[-2:]          # indices of the top-2 in this group
63         for k in keep:
64             m[g+k] = 1
65     return m
66
67 mask_mag = mask_24(mag_score)
68 mask_wanda = mask_24(wanda_score)
69 print("\n== 3. 2:4 masks (1 = keep, 0 = prune); two groups of 4 ==")
70 print("magnitude 2:4 mask =", mask_mag)
71 print("Wanda      2:4 mask =", mask_wanda)
72 print("kept weights (mag)  =", w * mask_mag)
73 print("kept weights (Wanda) =", w * mask_wanda)
74
75 # -----
76 # 4. Where do they disagree, and why? Show group 1 in detail.
77 # -----
78 print("\n== 4. Group 1 (features 0..3) detail ==")
79 for j in range(4):
80     print(f" f{j}: w={w[j]:+.2f} |w|={mag_score[j]:.2f} "
81           f" ||X||={col_norm[j]:.2f} Wanda={wanda_score[j]:.3f}")
82 print(" magnitude keeps top-2 by |w|  :", np.where(mask_mag[:4])[0])
83 print(" Wanda      keeps top-2 by score :", np.where(mask_wanda[:4])[0])
84
85 # -----
86 # 5. Why Wanda ~ Hessian importance (diagonal approximation).
87 # SparseGPT impact (Topic 1) is  $W^2 / [H^{-1}]_{jj}$ . Approximate H as
88 # diagonal:  $H_{jj} = ||X[:,j]||^2$ , so  $[H^{-1}]_{jj} \sim 1/||X[:,j]||^2$ , giving
89 # impact  $\sim W^2 * ||X[:,j]||^2 = (|W| * ||X[:,j]||)^2 = Wanda^2$ .
90 # So ranking by Wanda == ranking by the diagonal-Hessian impact score.
91 # -----
92 H_diag = (X**2).sum(axis=0)          #  $||X[:,j]||^2$  = diagonal of  $X^T X$ 
93 impact_diag = (w**2) * H_diag        #  $W^2 / (1/H_{jj}) = W^2 * H_{jj}$ 
94 print("\n== 5. Wanda approximates diagonal-Hessian impact ==")
95 print("diag(H)= $||X[:,j]||^2$       =", H_diag)
96 print("diag-Hessian impact  $W^2 * H$  =", impact_diag)
97 print("Wanda2                    =", wanda_score**2)
98 print("equal (up to fp)? ", np.allclose(impact_diag, wanda_score**2))
99
100 # -----
101 # 6. Memory / speed accounting for a 2:4 group.
102 # -----
103 print("\n== 6. 2:4 storage & speed accounting (one group of 4, FP16) ==")
104 dense_bytes = 4 * 2                  # 4 weights x 2 bytes
105 sparse_bytes = 2 * 2 + 1             # 2 values x 2B + 1 byte of indices
106 print(f"dense   group: 4 x 2B        = {dense_bytes} bytes")
107 print(f"2:4     group: 2 x 2B + index = {sparse_bytes} bytes")
108 print(f"bytes read ratio (dense/sparse) = {dense_bytes/sparse_bytes:.2f}x fewer-ish")
109 print(f"sparse tensor-core math speedup = 2.0x (half the MACs), ~1.8-1.95x measured")

```

— END OF DOCUMENT —